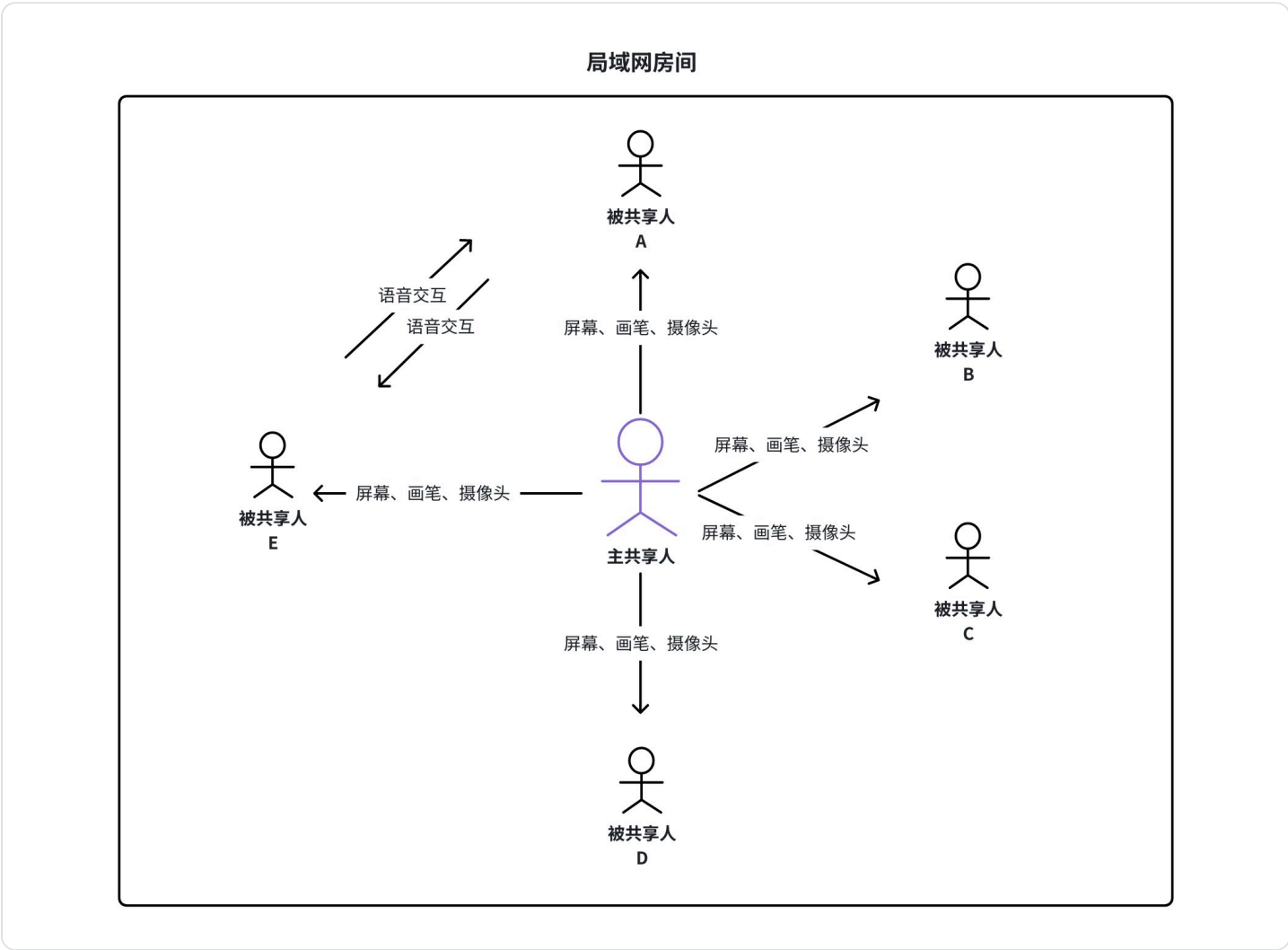


课程作业——实现一个局域网内的屏幕共享软件

一、课题描述

用c++语言开发一款客户端软件（支持windows、macos），实现在局域网内多人在线环境下的屏幕演示功能。即同一局域网内，多人登录该软件，其中一人点击共享屏幕的功能，能够将屏幕内容共享给其他所有人，其他所有人可以在自己的软件上看到共享人的屏幕内容。同时支持语音交互，主共享人在演示屏幕的时候可以打开麦克风进行语音讲解，其他成员听得到主共享人的说话，同时也可以打开自己的麦克风进行在线语音交流。也可以支持画中画功能，其他成员在观看主共享人演示屏幕的同时，也可以通过一个悬浮的小窗口区域看到主共享人的实时视频。也可以支持画笔标注功能，主共享人在屏幕区域通过软件提供的功能做标注，其他成员能够实时在自己的软件上看到标注内容。



二、课题要求

1、开发环境

开发语言：C++

开发平台：QT

开源框架：WebRtc

2、核心功能点

1. 视觉与交互

- a. 可以自行设计，这里不做额外的限定，可以自由发挥，实现核心功能即可，不要求非常精细

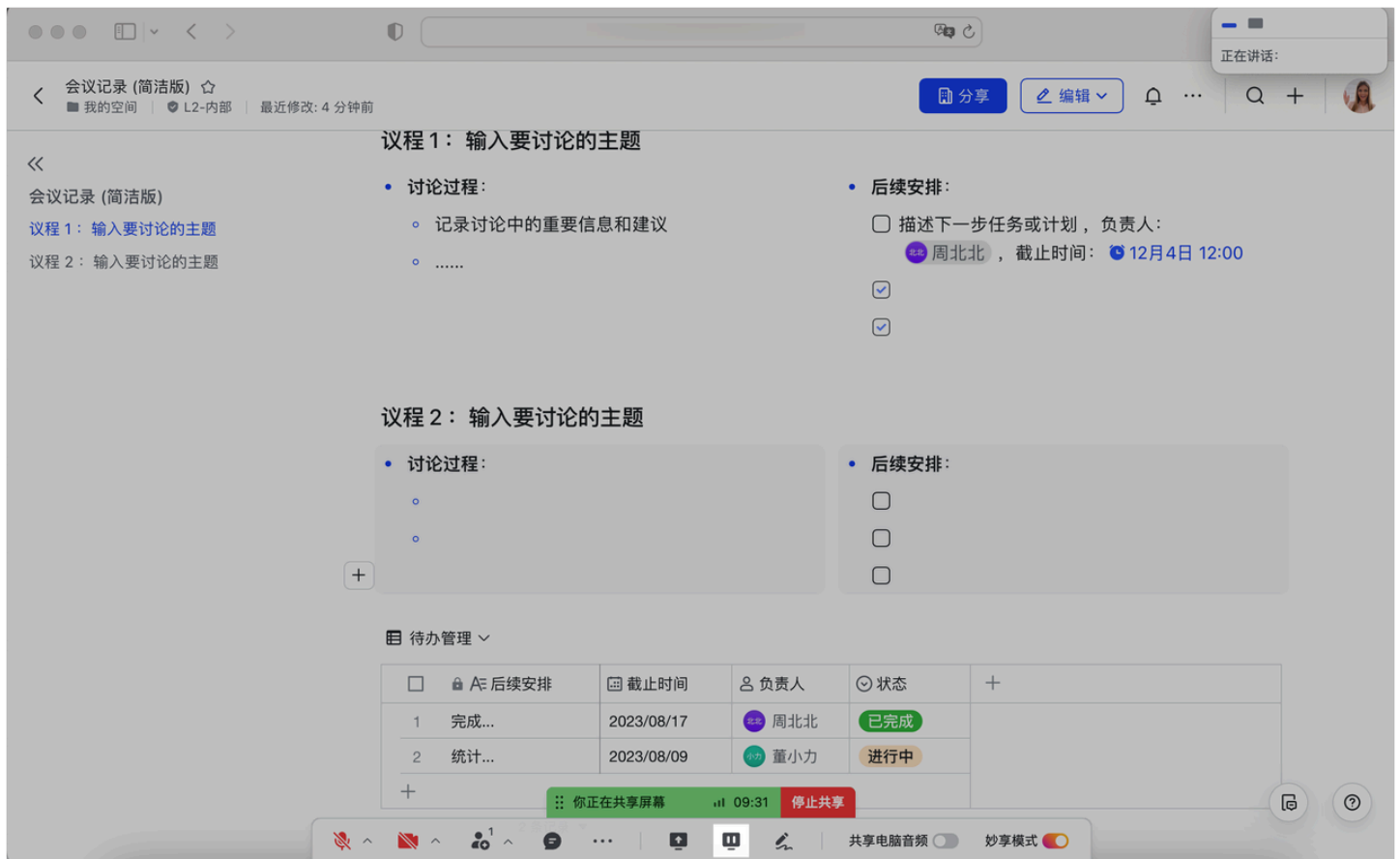
2. 房间加入与退出

- a. 登录软件后点击"加入房间"的按钮，进入到该局域网预设的一个房间内
- b. 登录软件后点击"退出房间"的按钮，退出房间不再接收屏幕流

3. 屏幕共享

- a. 用户通过点击"共享屏幕"的按钮可以将自己的屏幕同步给其他房间内的成员，其他房间内的成员可以在软件窗口内看到主共享人的实时屏幕。(可以参考飞书会议的屏幕共享功能)
- b. 用户通过点击"停止共享"的按钮可以停止当前的共享操作，其他的成员也看不到主共享人的屏幕
- c. 当主共享人在共享屏幕时，其他的成员可以通过点击"抢共享"按钮共享自己的屏幕，该成员将变成主共享人
- d. 语音传输，主共享人说话，其他被共享人能够听到

参考示意图



3、附加功能点(作为可选加分项，不要求必须完成)

1. 语音交互

- 房间内的所有成员都可以通过"打开"和"关闭"麦克风按钮，进行语音上的互动交流

2. 画中画

- 主共享人在打开自己的摄像头时，其他人可以看到共享人的视频画面，可以参考视频聊天，只有主共享人可以将自己的
- 设计上视频窗口可以通过一个小窗口的形式悬浮在共享屏幕的窗口之上。

3. 标注画笔（可以参考飞书视频会议的标注功能）

- 主共享人在共享屏幕的过程中，可以通过点击工具栏的标注功能，对屏幕视图进行画图操作，画图随即同步到其他成员

参考示意图



三、课题分工与进度

模块	负责人	职责	进度
信令交互			
屏幕共享模块 (win)	蒋宗原	窗口	
	韦燕丹	屏幕	
屏幕共享模块 (mac)	俞哲钊	窗口	
	邢雨茁	屏幕	
客户端业务架构 (mac/win)	黄俊杰 (mac)		
	赵梵年 (win)		
语音交互			

屏幕共享模块

week1:

开发环境熟悉/搭建

窗口或者屏幕枚举

自己分支开发

week2:

采集屏幕或者窗口，并和客户端同学联调

联调

week 1

- 确定角色和沟通方式
- 完成公共Git仓库的搭建，创建项目（github 或 gitee，推荐github）组长
- 完成研发环境设置（QT+WebRTC）
- 同类需求调研
- 本方案的技术选型、架构设计、流程设计
- 分析可能存在的问题（例如防火墙），分析一下这些问题是否影响未来开发进度
- 组件工作拆分（谁要做哪部分）、组件间依赖关系拆分、Milestone的设定
- 输出方案文档、进行技术评审、对技术方案熟悉的，和组内其他同学一起分享一下自己想要同步的内容
- 完成分工
- 建立一个属于自己的文档，遇到了什么问题，有什么新灵感，取得了什么新成果，都可以记录下来，你的答辩很有可能会要使用它
- 启动初步开发，编写README.md
- 第7天左右，完成Week 1的回顾

四、其他文档

方案：p2p连接的建立

下面给你一份**“纯 C++ / libwebrtc (Google 开源 WebRTC)” 的局域网 P2P 客户端示例**：用 **DataChannel** 做连通性验证，**手动粘贴 SDP** 完成信令交换（局域网里通常只靠 host candidate 就能直连）。WebRTC 本身不提供信令通道，你必须自己实现/选一种交换 SDP/ICE 的方式；这里用“复制粘贴”演示。(webrtc.googlesource.com)

关键点：

- DataChannel 想出现在 SDP 里，必须在创建 offer 前创建（否则 SDP 里没有 `m=application`）。(webrtc.googlesource.com)
- `SetLocalDescription(observer)` 可以让 WebRTC 自动创建 offer/answer 并设置本地描述（不 munging SDP）。(webrtc.googlesource.com)
- 解析/创建远端 SDP 用 `CreateSessionDescription(SdpType, sdp, &error)`。(webrtc.googlesource.com)

webrtc_lan_p2p_client.cpp（单文件可运行示例）

代码块

```
1  // webrtc_lan_p2p_client.cpp
2  // 用法：
3  //   A 机器：./webrtc_lan_p2p_client --role=offer
4  //   B 机器：./webrtc_lan_p2p_client --role=answer
5  //
6  // 交换方式：复制 A 输出的 SDP -> 粘贴到 B（以一行 END 结束）
7  //             复制 B 输出的 SDP -> 粘贴到 A（以一行 END 结束）
8  //
9  // 说明：这里为了“局域网最小 demo”，我们等待 ICE gathering complete 后再输出 SDP，
10 //      这样通常不用再单独粘贴 candidate（真实产品建议走 trickle ICE + 信令通道）。
11
12 #include <atomic>
13 #include <condition_variable>
14 #include <cstdlib>
15 #include <iostream>
16 #include <memory>
17 #include <mutex>
18 #include <optional>
19 #include <string>
20
21 #include "api/audio_codecs/builtin_audio_decoder_factory.h"
22 #include "api/audio_codecs/builtin_audio_encoder_factory.h"
23 #include "api/create_peerconnection_factory.h"
24 #include "api/data_channel_interface.h"
```

```

25 #include "api/jsep.h"
26 #include "api/peer_connection_interface.h"
27 #include "api/rtc_error.h"
28 #include "api/scoped_refptr.h"
29 #include "api/set_local_description_observer_interface.h"
30 #include "api/set_remote_description_observer_interface.h"
31 #include "api/video_codecs/builtin_video_decoder_factory.h"
32 #include "api/video_codecs/builtin_video_encoder_factory.h"
33
34 #include "rtc_base/copy_on_write_buffer.h"
35 #include "rtc_base/ref_counted_object.h"
36 #include "rtc_base/ssl_adapter.h"
37 #include "rtc_base/thread.h"
38
39 namespace {
40
41 std::string ReadMultilineUntilEND() {
42     std::string sdp;
43     std::string line;
44     while (std::getline(std::cin, line)) {
45         if (line == "END") break;
46         sdp.append(line);
47         sdp.push_back('\n');
48     }
49     return sdp;
50 }
51
52 void PrintSdpBlock(const std::string& title, const std::string& sdp) {
53     std::cout << "\n==== " << title << " (copy all, keep newlines) ==== \n";
54     std::cout << sdp;
55     if (!sdp.empty() && sdp.back() != '\n') std::cout << "\n";
56     std::cout << "==== END " << title << " ==== \n\n";
57 }
58
59 class DataChannelPrinter final : public webrtc::DataChannelObserver {
60 public:
61     explicit DataChannelPrinter(rtc::scoped_refptr<webrtc::DataChannelInterface>
dc)
62         : dc_(std::move(dc)) {}
63
64     void OnStateChange() override {
65         std::cout << "[DataChannel] state=" << static_cast<int>(dc_>state()) <<
"\n";
66     }
67
68     void OnMessage(const webrtc::DataBuffer& buffer) override {
69         std::string msg(buffer.data.data<char>(), buffer.data.size());

```

```

70     std::cout << "[Peer] " << msg << "\n";
71 }
72
73 void OnBufferedAmountChange(uint64_t /*sent_data_size*/) override {}
74
75 private:
76     rtc::scoped_refptr<webrtc::DataChannelInterface> dc_;
77 };
78
79 class SetLocalDescObserver final : public
webrtc::SetLocalDescriptionObserverInterface {
80 public:
81     using Callback = std::function<void(webrtc::RTCErrors)>;
82     static rtc::scoped_refptr<SetLocalDescObserver> Create(Callback cb) {
83         return rtc::scoped_refptr<SetLocalDescObserver>(
84             new rtc::RefCountedObject<SetLocalDescObserver>(std::move(cb)));
85     }
86     explicit SetLocalDescObserver(Callback cb) : cb_(std::move(cb)) {}
87     void OnSetLocalDescriptionComplete(webrtc::RTCErrors error) override {
cb_(std::move(error)); }
88 private:
89     Callback cb_;
90 };
91
92 class SetRemoteDescObserver final : public
webrtc::SetRemoteDescriptionObserverInterface {
93 public:
94     using Callback = std::function<void(webrtc::RTCErrors)>;
95     static rtc::scoped_refptr<SetRemoteDescObserver> Create(Callback cb) {
96         return rtc::scoped_refptr<SetRemoteDescObserver>(
97             new rtc::RefCountedObject<SetRemoteDescObserver>(std::move(cb)));
98     }
99     explicit SetRemoteDescObserver(Callback cb) : cb_(std::move(cb)) {}
100     void OnSetRemoteDescriptionComplete(webrtc::RTCErrors error) override {
cb_(std::move(error)); }
101 private:
102     Callback cb_;
103 };
104
105 enum class Role { Offer, Answer };
106
107 class LanPeer final : public webrtc::PeerConnectionObserver {
108 public:
109     explicit LanPeer(Role role) : role_(role) {}
110
111     ~LanPeer() override {
112         if (pc_) pc_>Close();

```



```

113     pc_ = nullptr;
114     factory_ = nullptr;
115
116     if (signaling_thread_) signaling_thread_>Stop();
117     if (worker_thread_) worker_thread_>Stop();
118     if (network_thread_) network_thread_>Stop();
119
120     rtc::CleanupSSL();
121 }
122
123 bool Init() {
124     if (!rtc::InitializeSSL()) {
125         std::cerr << "rtc::InitializeSSL() failed\n";
126         return false;
127     }
128
129     network_thread_ = rtc::Thread::CreateWithSocketServer();
130     worker_thread_ = rtc::Thread::Create();
131     signaling_thread_ = rtc::Thread::Create();
132
133     if (!network_thread_>Start() || !worker_thread_>Start() ||
!signaling_thread_>Start()) {
134         std::cerr << "Failed to start webrtc threads\n";
135         return false;
136     }
137
138     factory_ = webrtc::CreatePeerConnectionFactory(
139         network_thread_.get(),
140         worker_thread_.get(),
141         signaling_thread_.get(),
142         /*default_adm=*/nullptr,
143         webrtc::CreateBuiltinAudioEncoderFactory(),
144         webrtc::CreateBuiltinAudioDecoderFactory(),
145         webrtc::CreateBuiltinVideoEncoderFactory(),
146         webrtc::CreateBuiltinVideoDecoderFactory(),
147         /*audio_mixer=*/nullptr,
148         /*audio_processing=*/nullptr);
149
150     if (!factory_) {
151         std::cerr << "CreatePeerConnectionFactory failed\n";
152         return false;
153     }
154
155     webrtc::PeerConnectionInterface::RTCConfiguration config;
156     config.sdp_semantics = webrtc::SdpSemantics::kUnifiedPlan;
157
158     // 局域网最小 demo: 不配 STUN/TURN, 通常会生成 host candidates 直连。

```

```

159 // 若跨网段/有 NAT, 再加 STUN/TURN (或走 TURN relay) 。
160 // config.servers.push_back(...);
161
162 webrtc::PeerConnectionDependencies deps(this);
163 auto pc_res = factory_>CreatePeerConnectionOrError(config,
std::move(deps));
164 if (!pc_res.ok()) {
165     std::cerr << "CreatePeerConnectionOrError failed: " <<
pc_res.error().message() << "\n";
166     return false;
167 }
168 pc_ = pc_res.MoveValue();
169 return true;
170 }
171
172 bool CreateChatDataChannelIfOfferer() {
173     if (role_ != Role::Offer) return true;
174
175     webrtc::DataChannelInit init;
176     init.ordered = true;
177
178     auto dc_or = pc_>CreateDataChannelOrError("chat", &init);
179     if (!dc_or.ok()) {
180         std::cerr << "CreateDataChannelOrError failed: " <<
dc_or.error().message() << "\n";
181         return false;
182     }
183     data_channel_ = dc_or.MoveValue();
184     data_observer_ = std::make_unique<DataChannelPrinter>(data_channel_);
185     data_channel_>RegisterObserver(data_observer_.get());
186     return true;
187 }
188
189 // 触发创建 offer/answer 并设置 local description (由 WebRTC 根据当前 signaling
state 决定)
190 void StartSetLocalDescription() {
191     pc_>SetLocalDescription(SetLocalDescObserver::Create(
192         [this](webrtc::RTCErr err) {
193             if (!err.ok()) {
194                 Fail("SetLocalDescription", err);
195                 return;
196             }
197             local_desc_set_.store(true);
198             MaybeEmitLocalSdp();
199         }));
200 }
201

```

```

202     bool SetRemoteSdp(webrtc::SdpType type, const std::string& sdp) {
203         webrtc::SdpParseError parse_error;
204         auto desc = webrtc::CreateSessionDescription(type, sdp, &parse_error);
205         if (!desc) {
206             std::cerr << "CreateSessionDescription parse failed: "
207                 << parse_error.description << " | line: " << parse_error.line
208                 << "\n";
209             return false;
210         }
211
212         auto remote_type = desc->GetType();
213         std::mutex m;
214         std::condition_variable cv;
215         bool done = false;
216         bool ok = false;
217
218         pc_->SetRemoteDescription(
219             std::move(desc),
220             SetRemoteDescObserver::Create([&](webrtc::RTCErr err) {
221                 ok = err.ok();
222                 if (!err.ok()) {
223                     std::cerr << "SetRemoteDescription failed: " << err.message() <<
224                     "\n";
225                 }
226                 {
227                     std::lock_guard<std::mutex> lk(m);
228                     done = true;
229                 }
230                 cv.notify_one();
231
232                 // 如果收到的是 offer (answer 端) , 立刻创建并设置 answer (local) 。
233                 if (err.ok() && remote_type == webrtc::SdpType::kOffer) {
234                     StartSetLocalDescription();
235                 }
236             }));
237
238         std::unique_lock<std::mutex> lk(m);
239         cv.wait(lk, [&] { return done; });
240         return ok;
241     }
242
243     // 等待 ICE gathering complete 后输出“完整 SDP” (通常包含 candidates / end-of-
244     // candidates)
245     std::string WaitLocalSdp() {
246         std::unique_lock<std::mutex> lk(mu_);
247         cv_.wait(lk, [&] { return local_sdp_.has_value() || fatal_.load(); });
248         if (!local_sdp_) return "";

```

```

246     return *local_sdp_;
247 }
248
249 bool TrySend(const std::string& text) {
250     auto dc = data_channel_;
251     if (!dc) return false;
252     if (dc->state() != webrtc::DataChannelInterface::DataState::kOpen) return
false;
253
254     rtc::CopyOnWriteBuffer buf(text.c_str(), text.size());
255     webrtc::DataBuffer dbuf(buf, /*binary=*/false);
256     return dc->Send(dbuf);
257 }
258
259 bool HasDataChannel() const { return static_cast<bool>(data_channel_); }
260
261 // --- PeerConnectionObserver (必须实现纯虚函数) ---
262 void OnSignalingChange(webrtc::PeerConnectionInterface::SignalingState)
override {}
263 void OnAddStream(rtc::scoped_refptr<webrtc::MediaStreamInterface>) override
{}
264 void OnRemoveStream(rtc::scoped_refptr<webrtc::MediaStreamInterface>)
override {}
265 void OnRenegotiationNeeded() override {}
266
267 void
OnIceConnectionChange(webrtc::PeerConnectionInterface::IceConnectionState
new_state) override {
268     std::cout << "[ICE] conn_state=" << static_cast<int>(new_state) << "\n";
269 }
270
271 void OnIceGatheringChange(webrtc::PeerConnectionInterface::IceGatheringState
new_state) override {
272     if (new_state == webrtc::PeerConnectionInterface::kIceGatheringComplete) {
273         ice_complete_.store(true);
274         MaybeEmitLocalSdp();
275     }
276 }
277
278 void OnIceCandidate(const webrtc::IceCandidateInterface*) override {
279     // 真实产品一般这里把 candidate 发给对端 (trickle ICE)。
280     // 本 demo 走“等 gathering complete 后一次性发 SDP”，所以这里可不处理。
281 }
282
283 void OnDataChannel(rtc::scoped_refptr<webrtc::DataChannelInterface> channel)
override {
284     std::cout << "[DataChannel] received label=" << channel->label() << "\n";

```

```

285     data_channel_ = channel;
286     data_observer_ = std::make_unique<DataChannelPrinter>(data_channel_);
287     data_channel_->RegisterObserver(data_observer_.get());
288 }
289
290 private:
291     void MaybeEmitLocalSdp() {
292         // 注意: 官方接口说明 ToString 只能在 signaling thread 上安全调用, 我们放在回调里
293         // 触发。:contentReference[oaicite:4]{index=4}
294         if (!local_desc_set_.load() || !ice_complete_.load()) return;
295
296         std::lock_guard<std::mutex> lk(mu_);
297         if (local_sdp_.has_value()) return;
298         if (!pc_ || !pc_->local_description()) return;
299
300         std::string sdp;
301         if (!pc_->local_description()->ToString(&sdp) || sdp.empty()) return;
302
303         local_sdp_ = std::move(sdp);
304         cv_.notify_all();
305     }
306
307     void Fail(const char* where, const webrtc::RTCErrors& err) {
308         std::cerr << where << " failed: " << err.message() << "\n";
309         fatal_.store(true);
310         cv_.notify_all();
311     }
312
313 private:
314     Role role_;
315     std::unique_ptr<rtc::Thread> network_thread_;
316     std::unique_ptr<rtc::Thread> worker_thread_;
317     std::unique_ptr<rtc::Thread> signaling_thread_;
318
319     rtc::scoped_refptr<webrtc::PeerConnectionFactoryInterface> factory_;
320     rtc::scoped_refptr<webrtc::PeerConnectionInterface> pc_;
321
322     rtc::scoped_refptr<webrtc::DataChannelInterface> data_channel_;
323     std::unique_ptr<DataChannelPrinter> data_observer_;
324
325     std::atomic<bool> local_desc_set_{false};
326     std::atomic<bool> ice_complete_{false};
327     std::atomic<bool> fatal_{false};
328
329     std::mutex mu_;
330     std::condition_variable cv_;
331     std::optional<std::string> local_sdp_;

```

```

331 };
332
333 std::optional<Role> ParseRole(int argc, char** argv) {
334     for (int i = 1; i < argc; ++i) {
335         std::string a = argv[i];
336         if (a == "--role=offer") return Role::Offer;
337         if (a == "--role=answer") return Role::Answer;
338     }
339     return std::nullopt;
340 }
341
342 } // namespace
343
344 int main(int argc, char** argv) {
345     auto role_opt = ParseRole(argc, argv);
346     if (!role_opt) {
347         std::cerr << "Usage: " << argv[0] << " --role=offer|--role=answer\n";
348         return 1;
349     }
350
351     LanPeer peer(*role_opt);
352     if (!peer.Init()) return 2;
353
354     if (*role_opt == Role::Offer) {
355         // offer 端: 先建 data channel, 再 SetLocalDescription 生成 offer (避免 SDP 没
        有 m=application):contentReference[oaicite:5]{index=5}
356         if (!peer.CreateChatDataChannelIfOfferer()) return 3;
357         peer.StartSetLocalDescription();
358
359         auto offer_sdp = peer.WaitLocalSdp();
360         if (offer_sdp.empty()) return 4;
361         PrintSdpBlock("LOCAL OFFER SDP", offer_sdp);
362
363         std::cout << "Paste REMOTE ANSWER SDP, end with a line: END\n";
364         auto answer_sdp = ReadMultilineUntilEND();
365         if (answer_sdp.empty()) {
366             std::cerr << "No answer SDP\n";
367             return 5;
368         }
369         if (!peer.SetRemoteSdp(webrtc::SdpType::kAnswer, answer_sdp)) return 6;
370
371     } else { // Answer
372         std::cout << "Paste REMOTE OFFER SDP, end with a line: END\n";
373         auto offer_sdp = ReadMultilineUntilEND();
374         if (offer_sdp.empty()) {
375             std::cerr << "No offer SDP\n";
376             return 5;

```

```

377     }
378     if (!peer.SetRemoteSdp(webrtc::SdpType::kOffer, offer_sdp)) return 6;
379
380     // SetRemoteDescription 成功后会自动 StartSetLocalDescription() 生成 answer
    (见上面回调)
381     auto answer_sdp = peer.WaitLocalSdp();
382     if (answer_sdp.empty()) return 7;
383     PrintSdpBlock("LOCAL ANSWER SDP", answer_sdp);
384 }
385
386 std::cout << "=== Chat mode ===\n"
387             << "Type message and press Enter. Type /quit to exit.\n";
388 std::string line;
389 while (std::getline(std::cin, line)) {
390     if (line == "/quit") break;
391     if (!peer.TrySend(line)) {
392         std::cout << "[Info] DataChannel not open yet (or not created). Try
again.\n";
393     }
394 }
395 return 0;
396 }

```

你怎么用它（局域网两台机器）

1. A（offer 端）跑：

```
./webrtc_lan_p2p_client --role=offer
```

它会输出 LOCAL OFFER SDP

2. B（answer 端）跑：

```
./webrtc_lan_p2p_client --role=answer
```

把 A 的 SDP 粘贴进去，最后单独一行输入 END

3. B 会输出 LOCAL ANSWER SDP

把它粘贴回 A，最后输入 END

4. 两边进入 chat，能互发文本就说明 **P2P 通了**（底层是 ICE + DTLS + SCTP DataChannel）。

生产化你下一步要做什么（不啰嗦，但很关键）

- 把“复制粘贴 SDP”换成你自己的信令通道（WebSocket/HTTP/gRPC/UDP 广播都行），因为 WebRTC 的标准流程就是要交换 SDP 和（通常还要）ICE candidates。
(webrtc.googlesource.com)

- 如果你希望 **trickle ICE**（更快、更可靠），要在 `OnIceCandidate` 里把 candidate 发给对端，并在对端 `AddIceCandidate(...)`。(webrtc.google.com)

如果你告诉我你现在的目标是 **DataChannel** 还是 **音视频采集/推流**，以及你准备用什么做信令（比如 WebSocket），我可以把这个 demo 直接升级成“可自动互连”的两端代码（含最小信令协议）。

方案：抓取屏幕流并传输

在 C++ 里用 **libwebrtc** 做“屏幕采集 + WebRTC 发送给对端”，核心就是把这条链路接起来：

DesktopCapturer 抓屏 → 转成 I420(VideoFrame) → 自定义 VideoTrackSource → CreateVideoTrack → PeerConnection::AddTrack → 对端 OnTrack 渲染

下面给你一套可落地的骨架（省略你自己的信令/WebSocket实现、窗口渲染实现），结构尽量贴近 WebRTC 官方 native C++ 用法。

1. 你必须先有：信令 + ICE

WebRTC 不负责“找到对方并建立连接”，你需要自己做信令通道（WebSocket/HTTP/自定义 TCP 都行）交换：

- Offer / Answer SDP
- ICE candidates
- 使用 STUN/TURN（公网复杂网络强烈建议 TURN，比如 coturn）

`PeerConnectionInterface` 里也明确了 typical call 的步骤（创建 factory、PC、设置本地/远端 SDP、交换 candidate 等）。(webrtc.google.com)

2. 屏幕采集：DesktopCapturer（官方模块）

WebRTC 自带桌面采集模块 `modules/desktop_capture`，通过 `DesktopCapturer::CreateScreenCapturer(...)` 创建抓屏器。(chromium.google.com)

基本流程是：

1. `CreateScreenCapturer`
2. `GetSourceList` 拿屏幕/窗口列表
3. `SelectSource(source_id)`
4. `Start(callback)`

5. 定时调用 `CaptureFrame()` 控制帧率（比如 15/30 fps）

（抓屏是“你主动拉帧”，而不是摄像头那种自动推帧，这点在 `screen_capturer` 的注释里有强调完整 capture sequence）。(webrtc.googlesource.com)

3. 把 DesktopFrame 变成 WebRTC 可发送的视频源（关键点）

论坛里也说得很直白：你需要自己实现

`rtc::VideoSourceInterface<webrtc::VideoFrame>`（或用 `rtc::AdaptedVideoTrackSource`），底层用 `DesktopCapturer`，并把抓到的 RGB/ARGB 转成 I420。（[Google 组](#)）

下面是一个“能用的骨架”：`DesktopTrackSource` 同时做 **`DesktopCapturer::Callback`** 和 **`VideoTrackSource`**。

说明：`DesktopFrame` 常见是 BGRA/ARGB（平台相关），转换一般用 `libyuv`（WebRTC 源码树里自带 `third_party/libyuv`）。

代码块

```
1  // DesktopTrackSource.h
2  #pragma once
3  #include <atomic>
4  #include <memory>
5  #include <thread>
6
7  #include "api/video/video_frame.h"
8  #include "api/video/i420_buffer.h"
9  #include "api/scoped_refptr.h"
10 #include "rtc_base/ref_counted_object.h"
11 #include "rtc_base/synchronization/mutex.h"
12 #include "rtc_base/thread.h"
13 #include "rtc_base/time_utils.h"
14 #include "rtc_base/logging.h"
15 #include "rtc_base/task_queue.h"
16 #include "rtc_base/task_queue_for_test.h"
17
18 #include "modules/desktop_capture/desktop_capturer.h"
19 #include "modules/desktop_capture/desktop_capture_options.h"
20
21 // libyuv
22 #include "third_party/libyuv/include/libyuv/convert.h"
23
24 class DesktopTrackSource final
25     : public rtc::AdaptedVideoTrackSource,
26       public webrtc::DesktopCapturer::Callback {
27 public:
```

```

28     static rtc::scoped_refptr<DesktopTrackSource> Create(int fps = 15) {
29         return rtc::make_ref_counted<DesktopTrackSource>(fps);
30     }
31
32     explicit DesktopTrackSource(int fps)
33         : fps_(fps), running_(false) {}
34
35     // rtc::AdaptedVideoTrackSource
36     bool is_screencast() const override { return true; }
37     absl::optional<bool> needs_denoising() const override { return false; }
38     SourceState state() const override { return SourceState::kLive; }
39
40     // 启动采集：选择第一个屏幕（你也可以自己传 source_id）
41     bool StartCapture() {
42         webrtc::DesktopCaptureOptions options =
43         webrtc::DesktopCaptureOptions::CreateDefault();
44         capturer_ = webrtc::DesktopCapturer::CreateScreenCapturer(options); //
45         :contentReference[oaicite:4]{index=4}
46         if (!capturer_) return false;
47
48         webrtc::DesktopCapturer::SourceList sources;
49         if (!capturer_>GetSourceList(&sources) || sources.empty()) return false;
50
51         capturer_>SelectSource(sources[0].id);
52         capturer_>Start(this);
53
54         running_.store(true);
55         capture_thread_ = std::thread([this]() { this->CaptureLoop(); });
56         return true;
57     }
58
59     void StopCapture() {
60         running_.store(false);
61         if (capture_thread_.joinable()) capture_thread_.join();
62         capturer_.reset();
63     }
64
65     // DesktopCapturer::Callback
66     void OnCaptureResult(webrtc::DesktopCapturer::Result result,
67                         std::unique_ptr<webrtc::DesktopFrame> frame) override {
68         if (result != webrtc::DesktopCapturer::Result::SUCCESS || !frame) return;
69
70         const int width = frame->size().width();
71         const int height = frame->size().height();
72         const uint8_t* src = frame->data();
73         const int src_stride = frame->stride();

```

```

73 // 创建 I420 buffer
74 auto buffer = webrtc::I420Buffer::Create(width, height);
75
76 // 这里假设 frame 是 ARGB (不同平台可能是 BGRA/XRGB, 按实际改 libyuv 函数)
77 // 常见替代: libyuv::ARGBToI420 / BGRAToI420 / ABGRToI420 ...
78 libyuv::ARGBToI420(
79     src, src_stride,
80     buffer->MutableDataY(), buffer->StrideY(),
81     buffer->MutableDataU(), buffer->StrideU(),
82     buffer->MutableDataV(), buffer->StrideV(),
83     width, height);
84
85 webrtc::VideoFrame video_frame = webrtc::VideoFrame::Builder()
86     .set_video_frame_buffer(buffer)
87     .set_timestamp_us(rtc::TimeMicros())
88     .set_rotation(webrtc::kVideoRotation_0)
89     .build();
90
91 // 把帧推给 WebRTC track sinks
92 OnFrame(video_frame);
93 }
94
95 private:
96 void CaptureLoop() {
97     const int interval_ms = (fps_ <= 0) ? 66 : (1000 / fps_);
98     while (running_.load()) {
99         if (capturer_) capturer_->CaptureFrame(); // 主动拉帧控制帧率
100         std::this_thread::sleep_for(std::chrono::milliseconds(interval_ms));
101     }
102 }
103
104 int fps_;
105 std::atomic<bool> running_;
106 std::thread capture_thread_;
107 std::unique_ptr<webrtc::DesktopCapturer> capturer_;
108 };

```

注意：上面 ARGBToI420 的格式要按你平台的 `DesktopFrame` 实际像素格式调整；否则颜色会不对/花屏。

4. 把屏幕源挂到 PeerConnection（发送端）

WebRTC 官方 peerconnection C++ 示例里就是 `CreateVideoTrack(source)` 然后 `AddTrack(track, {streamId})`。(web rtc.google source.com)

骨架如下（省略了 PeerConnectionFactory/线程创建的样板、信令）：

代码块

```
1 // sender_side_snippet.cpp
2 #include "api/peer_connection_interface.h"
3 #include "api/create_peerconnection_factory.h"
4
5 #include "DesktopTrackSource.h"
6
7 void AddScreenTrack(
8     rtc::scoped_refptr<webrtc::PeerConnectionFactoryInterface> pcf,
9     rtc::scoped_refptr<webrtc::PeerConnectionInterface> pc) {
10
11     auto screen_source = DesktopTrackSource::Create(15);
12     if (!screen_source->StartCapture()) {
13         RTC_LOG(LS_ERROR) << "StartCapture failed";
14         return;
15     }
16
17     auto video_track = pcf->CreateVideoTrack(screen_source, "screen_track");
18     auto result = pc->AddTrack(video_track, {"stream_id"}); //
19     :contentReference[oaicite:7]{index=7}
20     if (!result.ok()) {
21         RTC_LOG(LS_ERROR) << "AddTrack failed: " << result.error().message();
22     }
23 }
```

5. 对端接收并渲染（接收端）

同样在官方示例里，对端收到 track 后在消息里 `StartRemoteRenderer(video_track)`。(web rtc.google source.com)

你需要在 `PeerConnectionObserver::OnTrack` / `OnAddTrack`（取决于你用的回调）里拿到 `VideoTrackInterface`，然后接到你的渲染器：

代码块

```
1 void OnTrack(rtc::scoped_refptr<webrtc::RtpTransceiverInterface> transceiver)
2     override {
3     auto track = transceiver->receiver()->track();
4     if (track->kind() == webrtc::MediaStreamTrackInterface::kVideoKind) {
```

```
4     auto video_track = static_cast<webrtc::VideoTrackInterface*>(track.get());
5     // 1) 接你的 Renderer/Sink
6     // video_track->AddOrUpdateSink(...)
7
8     // 2) 或者像官方示例那样交给窗口层 StartRemoteRenderer(video_track)
9     :contentReference[oaicite:9]{index=9}
10 }
```

渲染层实现看你用什么 UI:

- Qt: 把 I420 转 RGB 展示到 QWidget/QOpenGL
- SDL: YUV texture
- ImGui: 上传纹理

6. 常见坑（真实项目必踩）

- **权限**: macOS 需要“屏幕录制权限”；Windows/Wayland 也可能受系统策略影响。
- **Wayland**: Linux Wayland 下通常依赖 PipeWire, WebRTC 里也有判断是否运行在 Wayland。
(chromium.googlesource.com)
- **线程/生命周期**: 对端断开或 View 销毁时, 要 stop capture; 否则后台线程一直拉帧。
- **码率/分辨率**: 屏幕共享建议:
 - 限制 fps (10~15 很常用)
 - 开启分辨率适配 (缩放到 1280x720/1920x1080)
- **网络穿透**: 局域网还能侥幸直连, 公网强烈建议 TURN (否则大量网络环境会失败)。

如果你愿意, 我可以在你选定的平台 (Windows/macOS/Linux) 和 UI (Qt/SDL/ImGui/无 UI) 下, 把下面两块也补成“可编译运行”的最小工程:

1. 完整 `PeerConnectionFactory + Signaling(WebSocket)` 的可跑 Offer/Answer 示例
2. 接收端视频渲染 (YUV→RGB 显示) 和发送端屏幕源选择 (列出所有屏幕/窗口)